

Verstanden 😊. Die **vorletzte Antwort** war strukturierter und besser für die Präsentation. Dann ergänze ich genau diese Version mit den Namen der Personen.

Folie 1 – Roman

Beziehungen, Aggregation, Komposition und Multiplizitäten

Was zeigen wir?

Ausschnitt aus dem Klassendiagramm mit:

- Game
- Board
- Player
- Dice
- EstateSpace

Stichpunkte

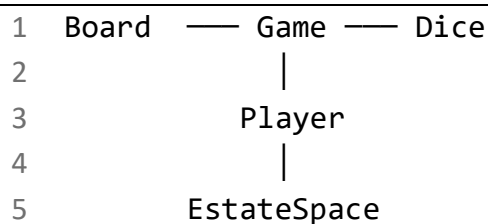
- Komposition: Game → Board, Game → Dice
- Komposition: Board → Space
- Aggregation: Game → Player
- Multiplizitäten (2..4 Spieler, 28 Felder)

Romans Sprechtext

Ich stelle die Beziehungen im Klassendiagramm vor. Das Game besitzt ein Board und einen Dice. Diese Beziehungen werden durch Komposition dargestellt. Außerdem besteht das Board aus 28 Spaces. Zwischen Game und Player haben wir eine Aggregation. Das bedeutet, dass das Spiel mehrere Spieler verwaltet. Die Multiplizitäten zeigen die Anzahl der Objekte. Zum Beispiel gibt es 2 bis 4 Spieler und ein Spieler kann mehrere Grundstücke besitzen.

Bild für diese Folie zuschneiden

Aus dem Klassendiagramm nur diesen oberen Bereich verwenden:



Folie 2 – Katharina

Kapselung und Static

Was zeigen wir?

Ausschnitt mit:

- Player
- Game
- Event (static)

Stichpunkte

- Private Attribute
- Getter und Setter
- Statische Eigenschaften und Methoden

Katharinas Sprechtext

Ich erkläre die Kapselung und die Verwendung von static. Attribute mit einem Minuszeichen sind privat und können nicht direkt von außen verändert werden. Der Zugriff erfolgt über Getter und Setter. Dadurch bleiben die Daten geschützt. Außerdem verwenden wir statische Elemente. Die Eigenschaft BoardSize in der Klasse Player ist für alle Spieler gleich. Auch die Klasse Event ist statisch. Ihre Methoden können direkt über die Klasse aufgerufen werden, ohne ein Objekt zu erzeugen.

Bild für diese Folie zuschneiden

Aus dem Klassendiagramm:

```
1  Game
2
3  Player
4
5  Event (static)
```

Besonders hervorheben:

```
1  - players: List<Player>
2  - currentPlayer: Player
```

```
1  Money { get; set; }
2  Name { get; }
```

```
1  BoardSize { static }
```

und die komplette Klasse

```
1  Event
```

Folie 3 – Babak

Vererbung, Abstrakte Klassen und Polymorphie

Was zeigen wir?

Ausschnitt mit:

- Space
- MoneySpace
- TaxSpace
- StartSpace
- EventSpace
- EstateSpace

Stichpunkte

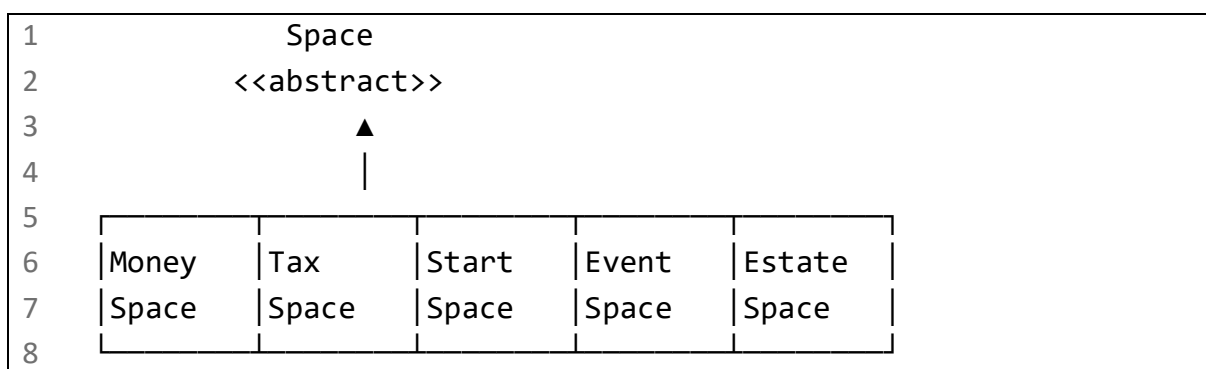
- Abstrakte Klasse Space
- Vererbung
- Polymorphie

Babaks Sprechtext

Ich erkläre die Vererbung und Polymorphie. Die Klasse Space ist eine abstrakte Oberklasse. Von ihr können keine Objekte erstellt werden. Die Klassen MoneySpace, TaxSpace, StartSpace, EventSpace und EstateSpace erben von Space. Alle Felder besitzen dadurch gemeinsame Eigenschaften und Methoden. Die Methode ExecuteAction wird in jeder Unterklasse unterschiedlich implementiert. Dadurch entsteht Polymorphie. Das Spiel kann mit dem Typ Space arbeiten und trotzdem wird zur Laufzeit automatisch die passende Methode des jeweiligen Feldes ausgeführt.

Bild für diese Folie zuschneiden

Nur den unteren Bereich des Diagramms:



Wichtig hervorheben:

- <<abstract>> Space
 - die Vererbungs-Pfeile
 - ExecuteAction(Player) in allen Unterklassen
-

Folie 4 – Team-Fazit

Projekt: Monopoly Konsolenspiel in C#

Aufteilung

- **Roman:** Beziehungen und Multiplizitäten
- **Katharina:** Kapselung und Static
- **Babak:** Vererbung, abstrakte Klassen und Polymorphie

Fazit

- ✓ Beziehungen zwischen Klassen
- ✓ Kapselung der Daten
- ✓ Vererbung gemeinsamer Eigenschaften
- ✓ Polymorphie durch ExecuteAction()
- ✓ Statische Elemente für globale Funktionen

Gemeinsamer Schlusssatz

Mit unserem Monopoly-Projekt haben wir die wichtigsten objektorientierten Konzepte in C# umgesetzt und praktisch in einem Konsolenspiel angewendet. Vielen Dank für Ihre Aufmerksamkeit. 😊